

PnPInk Documentation

None

None

Table of contents

1 PnPInk Documentation	7
2 Introduction	8
2.1 What is PnPInk?	8
2.2 The core idea	8
2.3 What you can do immediately	8
2.4 First contact: template, IDs, dataset	8
2.4.1 What is a template in PnPInk?	8
2.4.2 How IDs connect data to graphics	9
2.5 Minimal working example	9
2.5.1 Dataset notation used in this documentation	9
2.5.2 Conceptual template structure	9
2.5.3 Dataset example	9
2.6 A first taste of the DSL	10
2.7 What PnPInk can do (quick tour)	10
2.7.1 From simple repetition...	10
2.7.2 ...to data-driven variation	10
2.7.3 Precise layout control	11
2.7.4 Fronts and backs, automatically	11
2.7.5 Page-level elements	11
2.7.6 Fit and Anchor (single concept)	11
2.7.7 Adaptive layouts	11
2.7.8 Explicit ID arrays and slots	11
2.7.9 Slot-level alignment	11
2.7.10 Production features	12
2.8 A key practical advantage	12
2.9 Recommended reading path	12
Dataset	13
3.1 Dataset Format and Sources	13
3.1.1 Supported Inputs	13
3.1.2 Google Sheets Setup	13
3.1.3 Sheet Selection Logic	13
3.1.4 Dataset Section Structure	13
3.1.5 Why Column A Is Critical	14
3.1.6 Headers, Comments, and Directives	14
3.1.7 Multi-Dataset and Multi-Template	14

3.2 Dataset Reference	15
3.2.1 IDs and Naming	15
3.2.2 Dataset Structure	15
3.2.3 Header Types	16
3.2.4 Comments and Directives (#)	17
3.2.5 Leading Cell (column A in data rows)	17
3.2.6 @back -- Back-Side Templates (Back Pass)	18
3.2.7 @page -- Page-Anchored Templates (One Per Page)	18
3.2.8 Combining @page and @back	18
3.2.9 Advanced: Split Boards	19
4 ZVG and PNP Packages	20
4.1 Where They Appear in Inkscape	20
4.2 Import Extraction Folder	20
4.3 Package Contents	20
4.4 ZVG vs PNP Behavior	20
4.5 Compression Policy	21
4.6 Recommended Use	21
DSL	22
5.1 DSL Nomenclature	22
5.1.1 What belongs here	22
5.1.2 IDs and Targets	22
5.1.3 Module Form	22
5.1.4 Global Tokens	23
5.1.5 Lists, Ranges, and Repetition	23
5.1.6 Units and Expressions	24
5.1.7 Comment Directives and Declarations	24
5.1.8 Abbreviations, Defaults, and State Reuse	24
5.2 DSL Architecture	25
5.2.1 Data to SVG Pipeline	25
5.2.2 Front, Back, and Page Passes	25
5.2.3 Where Layout and Page Apply	25
5.2.4 Sources	25
5.2.5 Inkscape Integration	25
5.3 Iterators and Copies	27
5.3.1 Why this matters	27
5.3.2 Array Without Iterator ([...])	27
5.3.3 Iterator (*...)	27
5.3.4 Multi-Level Iteration (*, **, ***)	28

5.3.5 Supported Iterator Expressions	28
5.3.6 Copies in Column A (Card Quantity)	28
5.3.7 Examples	29
5.4 Fit-Anchor	30
5.4.1 Dataset Mental Model	30
5.4.2 What Is an "Object" Here?	30
5.4.3 Syntax and Style	30
5.4.4 Fit-Anchor Parameters	31
5.4.5 Compact Token Order (Reference)	33
5.4.6 Priority and Overrides	34
5.4.7 Practical Examples	34
5.4.8 Compact Notation (Clarifications)	34
5.5 Layout	36
5.5.1 Syntax	36
5.5.2 Pattern (p=)	36
5.5.3 Gaps (g=)	36
5.5.4 Offset (o=)	37
5.5.5 Shape (s=)	37
5.6 Page	38
5.6.1 Syntax	38
5.6.2 Size (default)	38
5.6.3 Landscape / Portrait	38
5.6.4 Border (b=)	38
5.6.5 Page Cursor (at / a / @)	38
5.7 Source	40
5.7.1 Syntax	40
5.7.2 Local File Sources	40
5.7.3 SVG Node Import (Optional)	41
5.7.4 Iconify Sources	41
5.7.5 Web Sources (HTTP/HTTPS)	41
5.7.6 Virtual Sources	41
5.7.7 Fit and Placement Behavior	42
5.7.8 Internal Model	42
5.7.9 Spritesheets (@)	43
5.8 Marks	44
5.8.1 Slot-Based Behavior	44
5.8.2 Syntax	44
5.8.3 Default Parameter (Style)	44

5.8.4 Length (len=)	44
5.8.5 Distance to Card (d=)	45
5.8.6 Border Pattern (b=)	45
5.8.7 Output Layer (layer=)	45
5.8.8 Scope	45
5.8.9 Examples	45
5.9 Gradients (Experimental)	47
5.9.1 Overview	47
5.9.2 Where to define gradients	47
5.9.3 How to apply gradients	47
5.9.4 Syntax	47
5.9.5 Stop definition	48
5.9.6 Behavioral details	49
5.9.7 End-to-end example	49
5.9.8 Current limitations	49
5.10 Advanced	50
5.10.1 Alias Definitions	50
5.10.2 Source Suffixes	50
5.10.3 Page Cursor Control (at)	50
5.10.4 Page Break Blocks	50
5.10.5 Leading Cell Composition	50
5.10.6 Inline Icon Tokens in Text	51
6 PnPInk User Manual	52
6.1 Introduction	52
6.2 What PnPInk Does	52
6.3 Key Principles	52
6.4 Power of Inkscape Integration	52
6.5 Intelligent Data and Automation	53
6.6 Extensible by Design	53
6.7 Output Quality and Professional Printing	53
6.8 Typical Use Cases	53
6.9 Why PnPInk	53
6.10 PnPInk Basic Workflow (User Guide)	54
6.10.1 What you need	54
6.10.2 Your first project: the quick overview	54
6.10.3 IDs: how elements are matched to data	54
6.10.4 Layouts & pagination (short notation)	54
6.10.5 Anchor & Fit (precise placement without manual nudging)	54

6.10.6 Inline icons (type the name, get the icon)	55
6.10.7 Snippets & variables (write once, reuse everywhere)	55
6.10.8 Asset sources (images, art, symbols)	55
6.10.9 Spritesheets and virtual sources (advanced, optional)	55
6.10.10 Style control without limits	55
6.10.11 Professional output (CMYK & print standards)	56
6.10.12 Typical end-to-end flow (checklist)	56
7 Basic Workflow	57
7.1 What you need	57
7.2 Create a Template	57
7.3 Prepare the Dataset	57
7.4 Define Page and Layout	57
7.5 Run DeckMaker	58
7.6 Useful Inkscape Panels for PnPInk	58
8 Snippets	59
8.1 What snippets are	59
8.2 Syntax	59
8.3 Arguments and Expansion	59
8.4 Defaults and Optional Parts	60
8.5 Quoting, Spaces, and Escaping	60
8.6 Nesting	60
8.7 Practical SVG Text Helpers	60
8.8 Processing Order	60
8.9 Summary Table	61
8.10 Design Principles	61
9 Presets	62
9.1 Page Presets (mm)	62
9.2 Shape Presets	63

1 PnPInk Documentation



PnPInk

Data-driven Print-and-Play

PnPInk is an Inkscape extension suite for data-driven print-and-play production.

Use the left navigation to browse chapters.

2 Introduction

2.1 What is PnPInk?

PnPInk is an open-source extension suite for Inkscape that turns it into a practical production environment for print-and-play components: cards, tiles, counters, boards, and player aids.

If you are new to Inkscape: Inkscape is a free, open-source vector editor based on the SVG standard. If you do not have it installed yet, you can download it for Windows, macOS, and Linux from the official website: <https://inkscape.org>

PnPInk works fully inside Inkscape. You design with normal SVG objects, and PnPInk handles replication, data filling, placement, and pagination automatically.

The output remains editable SVG, and you can export using Inkscape formats such as PDF, PNG, JPG, and SVG.

2.2 The core idea

PnPInk is built around a simple workflow:

1. Draw one component.
2. Describe variations in a dataset.
3. Let PnPInk generate the rest.

You start with a single visual design (a card, a tile, a token face), connect it to a dataset, and PnPInk produces as many instances as you need, placing them on pages and preparing them for printing.

You can begin with defaults. Advanced controls are optional and can be added gradually when you need more precision.

2.3 What you can do immediately

Without advanced syntax, you can already:

- duplicate one template many times,
- change texts and images per instance,
- auto-fill pages,
- generate multi-page output ready to export.

Everything beyond that is incremental.

2.4 First contact: template, IDs, dataset

2.4.1 What is a template in PnPInk?

A template is a normal Inkscape drawing that represents one unit to replicate: one card, one tile, one board section, and so on.

In practice, it is a group of regular SVG objects (text, rects, paths, images, groups), identified by IDs.

PnPInk uses one internal object as template bounding box (`bbox`) to understand:

- template size,
- placement reference,
- replication behavior.

The `bbox` can be a rect or another simple shape. What matters is that its outline correctly wraps the component you want to replicate.

2.4.2 How IDs connect data to graphics

In PnPInk, IDs are the connection between dataset and drawing.

If a dataset column is named `title` and your SVG contains `id="title"`, that object can be updated for each generated row.

Typical usage:

- text IDs: dynamic text replacement,
- rect IDs: image/icon anchors,
- group IDs: visibility or variant control (advanced workflows).

Useful Inkscape panels:

- `Object > Objects...` (`Shift+Ctrl+O`): hierarchy, groups, layers, Z-order, IDs.
- `Object > XML Editor` (`Shift+Ctrl+X`): direct SVG/XML attributes.

2.5 Minimal working example

2.5.1 Dataset notation used in this documentation

To avoid confusion, this guide uses a consistent visual notation:

- header cells: dataset column names (first row),
- regular cells: normal data values,
- first-column cells: cells in column 1 (template/page/layout control).

In dataset tables, headers and first-column cells are also color-highlighted.

2.5.2 Conceptual template structure

Use a basic Inkscape file named `hello_word.svg`.

In Inkscape, the structure can look like this:

```
(g) hello_word_template
|- (rect) card_bbox      <- template bounding box
|- (text) title
|- (text) cost
|- (rect) art            <- image/icon anchor
|- (text) text
```

Key points:

- `card_bbox` visually wraps the full component.
- `title`, `cost`, `art`, and `text` are the objects you vary per row.

2.5.3 Dataset example

Spreadsheet-like table view:

<code>card_bbox</code>	<code>title</code>	<code>cost</code>	<code>art</code>	<code>text</code>
{A4 b=[-5]}.L{p=4x3 g=2}	Tomatoes	3	tomato	You win 1 tomato
	Mushrooms	5	brown-mushroom	You win 2 mushrooms
	Lemons	2	lemon	Win 1 lemon for every tomato you own

CSV text view (same data):

```
card_bbox,title,cost,art,text
{A4 b=[-5]}.L{p=4x3 g=2},Tomatoes,3,tomato,You win 1 tomato
,Mushrooms,5,brown-mushroom,You win 2 mushrooms
,Lemons,2,lemon,Win 1 lemon for every tomato you own
```

Interpretation:

- The header of the first column is `card_bbox`.
- The first-column cell `{A4 b=[-5]}.L{p=4x3 g=2}` sets page/layout context for this dataset block.
- Empty first-column cells continue using the same template/page context.
- Each row generates one component instance.
- `title` and `cost` update text fields.
- `art` provides the image source for the `art` anchor.
- `text` updates the text object with `id="text"`.

With defaults only, PnPink places instances sequentially, fills the page, and creates additional pages automatically when needed.

2.6 A first taste of the DSL

Once basics work, you can start controlling page and layout with a short expression:

```
{A4 b=[-5]}.L{p=4x3 g=2}
```

At a glance:

- `{A4 b=[-5]}`: A4 page with 5 mm inner margin at every side.
- `.L{p=4x3 g=2}`: layout of cards in a pattern of 4x3 grid with 2 mm gap.

This short notation is part of the PnPink DSL (Domain Language). It lets you control placement, scaling, rotations, grids, gaps, bleeds, marks, and more.

PnPink is designed to be simple by default, and powerful when you need it.

2.7 What PnPink can do (quick tour)

Even if you do not understand every syntax detail yet, this gives you a practical map of what is possible.

2.7.1 From simple repetition...

Build many components from one design and one dataset:

```
{A4}.L{3x4}
```

One template can be placed 12 times on an A4 page. See [Layout](#) and [Page](#).

2.7.2 ...to data-driven variation

Each dataset row can produce a different result. Texts, images, icons, and properties can vary per instance. See [Dataset Reference](#).

2.7.3 Precise layout control

Control spacing and sizing explicitly:

```
L{3x4 gaps=4 shape=poker}
```

See [Layout](#).

2.7.4 Fronts and backs, automatically

Generate aligned duplex backs with `@back` :

```
{card_back @back}
```

See [DSL Advanced](#).

2.7.5 Page-level elements

Place objects once per page, not once per card:

```
{page_title @page}
```

Useful for titles, page numbers, frames, or static page backgrounds. See [Page](#).

2.7.6 Fit and Anchor (single concept)

Position objects relative to target rectangles without manual coordinates:

```
icon.F{i a=9}
```

The element is fitted and anchored by intent, not by absolute measurements. See [Fit and Anchor](#).

2.7.7 Adaptive layouts

Let layout adapt to available space:

```
L{1x? gaps=?}
```

Items stack and spacing is computed automatically. See [Layout](#).

2.7.8 Explicit ID arrays and slots

Apply layout directly to selected IDs:

```
[id1 id2 - id3].L{1x?}
```

- reserves an empty slot without rendering. See [Core Syntax](#).

2.7.9 Slot-level alignment

Align content inside each layout slot:

```
L{1x? a=6}
```

See [Layout](#) and [Fit and Anchor](#).

2.7.10 Production features

Generate cut marks aligned with final geometry:

```
.M{len=[3 2] d=2}
```

Marks follow real layout, spacing, bleeds, and rotations. See [Marks](#).

Use external sources (images, PDFs, spritesheets, icon libraries), and reuse generated assets in pipelines. See [Source](#).

2.8 A key practical advantage

With PnPink you do not need to work with absolute coordinates for most production tasks.

You can express intent, for example: bring an image from a source, rotate it, fit it to a top-right anchor, scale it, and crop overflow.

If the source image changes, the same rule still works. If you switch card size (for example from poker to tarot), layout and fitting scale with the template logic, without manually re-measuring everything.

This is one of the main differences between PnPink and many manual or coordinate-heavy workflows.

2.9 Recommended reading path

1. [Basic Workflow](#)
2. [Dataset Format and Sources](#)
3. [Dataset Reference](#)
4. [DSL Nomenclature](#)
5. [DSL Modules](#)
6. [Fit and Anchor](#)

Dataset

3.1 Dataset Format and Sources

PnPink reads tabular data from CSV files or Google Sheets and converts it into one or more dataset sections.

3.1.1 Supported Inputs

- Local CSV file.
- Google Sheet (`sheet_id` in DeckMaker).

If `sheet_id` is empty, DeckMaker loads CSV from the same folder as the SVG:

```
<svg_name>.csv
```

3.1.2 Google Sheets Setup

In `Extensions > PnPink DEV > DeckMaker v0.24dev`, fill:

- `Google Sheet Id (optional)`
- `Range (A1, optional)`

How to get `sheet_id`:

- From a Google Sheets URL like: `https://docs.google.com/spreadsheets/d/<SHEET_ID>/edit#gid=0`
- Use the part between `/d/` and `/edit`.

3.1.3 Sheet Selection Logic

When `sheet_id` is set:

1. If `Range` includes a sheet name (`Sheet2!A1:Z999`), that sheet is used.
2. Otherwise, PnPink searches a sheet whose title matches the SVG filename (without extension), case-insensitive.
3. If no match is found, it uses the first sheet.

Default range is `A1:Z999` if empty.

This lets one Google Spreadsheet host multiple projects (one tab per SVG).

3.1.4 Dataset Section Structure

Each dataset section has:

- column A: section marker and row-level control cell,
- columns B+: headers and data fields.

Two modes are supported:

- Marker mode: `{{...}}` in column A (recommended, supports multiple dataset sections).
- Shorthand mode: single dataset; first row uses A1 as main bbox id.

See full syntax in [Dataset Reference](#).

Minimal Dataset Example

Column A	title	cost	art
card_bbox			
	Fireball	3	images/fireball.png
	Shield	2	images/shield.png

```
card_bbox,title,cost,art
,Fireball,3,images/fireball.png
,Shield,2,images/shield.png
```

3.1.5 Why Column A Is Critical

The first bbox id (`t=...` or equivalent) in column A defines the main template for that section. It drives:

- layout slots and pagination,
- marks generation,
- front/back slot pairing,
- page-membership selectors for `@page`.

3.1.6 Headers, Comments, and Directives

Headers and comments are part of the dataset grammar, not free text. Before authoring complex sheets, review:

- [Dataset Reference](#) for exact rules,
- [Snippets](#) for `# :Name(...) = ...` directives,
- [DSL Nomenclature](#) for shared token rules.

3.1.7 Multi-Dataset and Multi-Template

- Multiple datasets in one sheet are supported via repeated marker rows (`{{...}}`).
- Multiple template columns are supported via header declarations (`{template_id ...}`).
- Multiple main templates in one marker list are not supported; use one main template per dataset section.

3.2 Dataset Reference

This chapter is the exact behavioral reference for dataset parsing and execution. Use [Dataset Format and Sources](#) first if you are new to the model.

3.2.1 IDs and Naming

This section explains how table headers bind to SVG content.

Dataset headers are matched to SVG IDs.

For the Inkscape ID workflow (where to inspect and edit IDs), see [Introduction -> How IDs connect data to graphics](#).

IDs must be unique and follow XML rules (letters first, no spaces).

3.2.2 Dataset Structure

Dataset structure defines where parsing starts and how sections are separated.

PnPink supports two forms:

- **Marker mode** (recommended, supports multiple datasets).
- **Shorthand mode** (single dataset only).

For source selection (CSV vs Google Sheets and tab lookup), see [Dataset Format and Sources](#).

Marker Mode (column A)

Marker mode is the robust format for production datasets and multi-section files.

A dataset marker exists **only in column A** and uses `{{...}}`. The marker row is also the header row. Headers start in column B.

Examples (equivalent):

```
{{t=card_bbox}}
{{template_bbox=card_bbox}}
{{card_bbox}}
```

Notes:

- Only one main template bbox is supported per dataset marker.
- Extra DSL tails are allowed after the marker (see Leading Cell below).

Main Template BBox and Z-Order

This is a core concept: one main bbox drives slot logic for the section.

The marker `t=...` defines the **main template bbox** for that dataset section. This main template controls:

- slot planning (`Layout{}` / `Page{}`),
- per-slot marks (`Marks{}`),
- front/back slot pairing (`@back`),
- page membership for page-anchored templates (`@page` selectors).

Placement order (Z-order) follows dataset row order. Later rows are rendered above earlier rows.

Shorthand Mode (single dataset)

Shorthand mode is a convenience form for small, single-section sheets.

If there is no marker row, the first non-empty, non-comment row is treated as the header row.

In shorthand mode, column A contains the template bbox id:

```
card_bbox, title, cost, art
```

This is equivalent to:

```
{{t=card_bbox}}, title, cost, art
```

3.2.3 Header Types

Header type determines whether a column edits fields or instantiates templates.

Headers in column B+ can be:

1. **Normal data fields:** match SVG IDs and replace text or sources.
2. **Template columns:** declare extra templates with `{...}`.

Normal Data Field Syntax

Use this syntax for the most common per-row updates (text, source, and defaults).

Headers can include modifiers:

```
title
art+
price[xml]
bg=default_bg
art=art_placeholder-i5
```

Rules:

- `id+` keeps the original anchor rect visible (otherwise anchors are hidden).
- `id[prop]` sets the property (`text` or `xml`). Default is `text` .
- `id=...` declares a **default value** or default Fit ops for that column.
- `id-*` in headers expands to all matching IDs by prefix.
- `id1 id2 id3` in a single header applies the same cell value to all listed IDs.

Examples:

```
id=default_id
id=~i5
id=default_id-i5
```

Header Fan-Out and Wildcards

Use this when one dataset column must feed several placeholders.

Examples:

```
ph-1 ph-2 ph-3
id1
```

`id1` is applied to all three placeholders.

Wildcard headers are also supported:

```
main_icon-*
Ic(heart-suit)
```

This applies the value to every placeholder whose ID starts with `main_icon-`. All normal header modifiers (`+`, `[xml]`, `=...`) remain valid.

Template Column Syntax

Use template columns when you need extra template instances, back passes, or page-level elements.

Template columns declare template bbox IDs and modifiers:

```
{card_back @back}
{page_title @page}
{back_bg @page @back}
```

Supported modifiers:

- `@page` page-anchored templates
- `@back` back-pass templates

Template columns are rendered as additional instances; they do not replace the main template column logic.

3.2.4 Comments and Directives (#)

Comments are processed **before any other operation**. Rules are global (same behavior everywhere; no inside/outside distinction).

- `#` at line start (first non-space char in column A): comment/directive row.
- `##` at line start: full comment row (not a directive).
- `####` at line start: EOF marker, stops parsing the rest of the file/sheet.
- `##` inside a cell: comments out the rest of that cell.
- `####` inside a row (not line start): comments out the rest of that cell and all cells to the right (rest of line).
- Single `#` inside a cell is normal text (not a comment).

Header disabling still works:

- `##header` disables that column.
- `###header` disables that column and all columns to the right.

3.2.5 Leading Cell (column A in data rows)

Leading-cell directives are row-level controls, not regular data fields.

Column A in data rows can carry row-level DSL:

- `{A4 ...}` page block
- `L{...}` layout tail
- `M{...}` marks tail
- trailing copies number
- optional hole patterns in `[...]`

Examples:

```
{A4 b=[-5]} L{p=3x3 g=2} M{mk_cut} 2
[3 - 2-]          -> 3 copies, then 1 hole, then 2 holes
```

This cell is **not** a normal dataset field; it controls row-level layout/flow. Its directives apply before regular field replacements in that row.

If a row uses only column-A controls and all payload cells (columns B+) are empty, PnPink applies the controls but does **not** generate a card/instance for that row.

```
card_bbox,title,cost
{A4},,
,Fireball,3
```

3.2.6 @back -- Back-Side Templates (Back Pass)

Use `@back` for duplex output where back faces must align with front slot geometry.

A template column marked with `@back` is rendered **only on back pages**.

```
{card_back @back}
```

Back templates:

- reuse the same slot sequence from the front layout,
- mirror slot placement within the page for duplex alignment,
- preserve row order as Z-order.
- back pages are inserted right after each front page (interleaved).

If a dataset cell for an `@back` column is empty or contains `-` or `0`, that back instance is skipped.

3.2.7 @page -- Page-Anchored Templates (One Per Page)

Use `@page` for elements that belong to the page frame, not to per-card slots.

A template column marked with `@page` is anchored to the **page frame**, not to the slot grid.

```
{page_title @page}
```

Page-anchored templates:

- are positioned relative to the page frame after `Page{}` margins,
- are rendered once per page,
- use Fit/Anchor for placement.

Each dataset row provides a **slot selector** in the cell value (e.g. `-8[-5]`). The slot determines which page the template belongs to; the rest is Fit/Anchor ops.

Selectors can be a single slot index, range, or list:

```
-1
-[1 3 5]
-[2..4]
```

If multiple rows target the same page, only the first is rendered (a warning is logged).

3.2.8 Combining @page and @back

Combining both modifiers is common for back-page backgrounds and page-level back labels.

```
{back_bg @page @back}
```

This places a page-anchored element on back pages, aligned to the front page sequence.

3.2.9 Advanced: Split Boards

This mode is activated automatically when template dimensions exceed target page inner size.

If the template is larger than the target page, the engine switches to split-board mode:

- the template is cut into tiles,
- each tile is placed on a page,
- layout and marks are applied to the tile bounds.

4 ZVG and PNP Packages

PnPink supports two ZIP-based package formats for portable projects:

- `.zvg`: package the current SVG project and local assets for reproducible sharing.
- `.pnp`: package a lightweight project intended to regenerate content from dataset/web sources on open.

4.1 Where They Appear in Inkscape

- `File > Open` can open `.zvg` and `.pnp`.
- `File > Save As / Save a Copy` can export `.zvg` and `.pnp`.

4.2 Import Extraction Folder

When opening a `.zvg` or `.pnp`, PnPink extracts package contents into a dedicated folder:

- folder path: same directory as the package file,
- folder name: package filename without extension.

Example:

- package: `MyDeck.pnp`
- extracted to: `MyDeck/` (contains SVG, manifest, CSV, assets, etc.)

This keeps assets from different packages isolated and avoids cross-package mixing.

4.3 Package Contents

Typical package structure:

- One main SVG in ZIP root.
- Optional CSV dataset.
- Optional `manifest.json`.
- Local assets (typically under `assets/`).

Defaults work without manifest:

- If there is one SVG in root, it is used as main document.
- CSV is expected with the same base name as the SVG.
- Assets are expected under `assets/`.

If multiple SVG files are included, set `manifest.json` with the `svg` field to disambiguate.

4.4 ZVG vs PNP Behavior

- `.zvg` favors reproducibility:
 - includes referenced local assets,
 - includes local cached downloadable assets when referenced.
- `.pnp` favors minimal payload:
 - includes required local project assets,

- skips cache-like downloaded files when possible,
- if local CSV is missing and `gsheet_id` is available (options or manifest), export generates a CSV snapshot from Google Sheets and includes it,
- may run DeckMaker on import when requested by manifest (`run_deckmaker_on_import`).

4.5 Compression Policy

- Images and PDFs are stored without deflate recompression:
 - `.png` , `.jpg` , `.jpeg` , `.webp` , `.pdf`
- Other files are compressed with deflate (level 9).

4.6 Recommended Use

- Use `.zvg` to archive/share an exact visual state.
- Use `.pnp` to share a compact, regeneration-oriented project.

DSL

5.1 DSL Nomenclature

This chapter defines the shared language used across the DSL (Domain Language). Use it as the common reference before reading module-specific pages.

5.1.1 What belongs here

PnPInk has two complementary parts:

- dataset structure (rows, headers, section markers, control columns),
- DSL logic (declarative syntax that controls how data is interpreted and rendered).

This chapter is about the second part: the DSL itself.

In practice, this includes:

- module syntax (`Fit`, `Layout`, `Page`, `Source`, `Marks`, `Gradients`),
- compact notation (`~...` and operator shorthands),
- list/range/repetition grammar,
- global directives in comment lines (`# ...`) such as snippets, spritesheet aliases, and other declarations.

For dataset structure and section markers, see [Dataset Reference](#).

5.1.2 IDs and Targets

An SVG `id` is the primary selector for DSL operations.

For the Inkscape ID workflow (where to inspect/edit IDs and structure), use: [Introduction -> How IDs connect data to graphics](#).

Target forms:

- one SVG object id: `title`,
- grouped target list: `[id1 id2 id3]`,
- source target: `@{assets/icon.svg}`.

These selectors let one logical target expose multiple concrete items without creating many separate headers.

5.1.3 Module Form

Modules define behavior declaratively. Use long form for clarity and short form for compact production datasets.

Long form:

```
target.Module{ key=value key2=value2 }
```

Here, `key` and `key2` are module **parameters**. Each module defines its own valid parameter set, defaults, and aliases. In this documentation, we always call them **parameters** (not properties).

Short form:

```
target.M{ ... }    # module short name
target~...         # Fit-Anchor shorthand
```

Common modules:

- `Fit{}` / `~...`
- `Layout{}` / `L{}`
- `Page{}`
- `Source{}` / `@{}`
- `Marks{}` / `M{}`

5.1.4 Global Tokens

- `[]` list or list-like values.
- `{}` module blocks (`Page`, `Layout`, `Marks`, etc.).
- `^` rotation (and page landscape in page tokens).
- `|` and `||` mirror (horizontal, vertical).
- `!` clip (see Fit-Anchor clip stage details).
- `:` substitution (inline `:icon:` and `:snippet(...)`).
- `*` has three different uses depending on context:
- wildcard in identifiers/paths (`id*`, `file*.png`),
- iterator prefix in dataset values (`*[ ... ]`, `**[ ... ]`, ...),
- repetition inside list/array bodies (`3*id`, `3*(id1 id2)`).
- `()` function arguments (`:snip(a b)`) and group blocks inside lists/iterators.

5.1.5 Lists, Ranges, and Repetition

Lists are space-separated:

```
[ID1 ID2 ID3]
```

Supported patterns in list/array bodies:

```
[ID1 - - - ID4]    -> same as [ID1 3- ID4], '-' means empty slot
3*ID               -> [ID ID ID]
5*:Ic(potato)      -> [:Ic(potato) :Ic(potato) :Ic(potato) :Ic(potato) :Ic(potato)]
[id1 3*(id2 5- id3)]
```

Repetition:

- `n*Id` means repeat `Id` `n` times.
- Example: `[3*Id]` is equivalent to `[Id Id Id]`.
- `Id` can be a `( )` grouped block: `[2*(idA idB)]` is equivalent to `[idA idB idA idB]`.

Ranges and selectors:

```
target[2]          -> item 2
target[2..4]       -> items 2, 3, 4
target[A..ZZ]      -> A B C ... Z AA AB .. ZZ
target[*]          -> all items
target[2][3]       -> chained selector for nested/indexed structures
```

Note that `target[*]` (selector) and `*[ ... ]` (iterator) are not the same operation. - `target[*]` selects all items inside one target expression. - `*[ ... ]` expands rows/instances during iterator processing (assign items to different cards)

5.1.6 Units and Expressions

In paramters, most numeric tokens accept:

- plain numbers (current default unit),
- `mm`, `px`, `%`,
- mixed expressions such as `-25%+2`.

Percentages are evaluated against a base size. For example `border=[80% 110%+3]` define a new rect size with 80% of actual width, and 110% of height + 3 mm.

5.1.7 Comment Directives and Declarations

Comment rows (`# ...`) can declare DSL directives (aliases, variables, spritesheets...) outside normal data rows. Typical examples:

```
# :snippet(x) = ...
# @sp1 = @{sheet.png}.Layout{8x6}
# gradientGold = G{b88326ff^25 [-25 12 5] [55 35 20 10]}
# _DM_reset_to=3
```

These declarations are read by their corresponding modules/subsystems and then reused in dataset rows.

5.1.8 Abbreviations, Defaults, and State Reuse

Keyword abbreviations are supported with the firs character.

```
Layout{} -> L{}
inside -> i
Page{pagesize=A4 border=[-2]} -> P{A4 b=[-2]}
```

Most modules define default parameters, so compact forms can omit explicit keys. I.e. you dont need to specify `p=A4` because "pagesize" is the default parameter por `Page{}` module.

When a required value is omitted, the last active state may be reused (module-dependent behavior).

For exact page-state rules, see [Page](#). For Fit-Anchor shorthand order and precedence, see [Fit-Anchor](#).

5.2 DSL Architecture

High-level flow of how PnPInk applies the DSL.

5.2.1 Data to SVG Pipeline

This sequence explains where each DSL family acts in the generation process.

PnPInk processes in this order:

1. Read dataset (CSV or Google Sheets).
2. Parse dataset marker and header row.
3. Clone the template for each row (main template column).
4. Resolve sources (images/icons/URLs) into SVG symbols.
5. Apply Fit/Anchor to place each element.
6. Apply Layout to place instances into slots and pages.
7. Render Marks (cut marks) per placed slot.

5.2.2 Front, Back, and Page Passes

These passes are execution phases, not just syntax flags. They define when and where templates are rendered.

Header modifiers control when a template is rendered:

- `@page` : page-anchored, once per page, positioned against the page frame.
- `@back` : rendered in the back pass, aligned to front slots.
- `@page @back` : page-anchored but rendered on back pages.

These modifiers belong to template headers, not to data cells.

5.2.3 Where Layout and Page Apply

This distinction prevents common mistakes when debugging placement.

- `Page{}` defines page size, margins, and cursor position.
- `Layout{}` defines the slot grid and how instances are placed.
- `Fit{}` defines how elements sit inside their anchor rects.

Page state is global. Layout state is per dataset. Fit is per element.

5.2.4 Sources

Sources are resolved once and then treated as normal placement targets.

`Source{}` / `@{}` creates a reusable SVG symbol and then places it with Fit, so sources behave like normal SVG targets.

5.2.5 Inkscape Integration

These panels are the operational bridge between visual authoring and dataset-driven generation.

PnPInk relies on Inkscape objects and IDs:

- IDs and structure workflow: [Introduction -> How IDs connect data to graphics](#).
- Symbols: [Object > Symbols](#) (Shift+Ctrl+Y).
- Layers: [Layer > Layers](#) (Shift+Ctrl+L).

5.3 Iterators and Copies

This chapter explains the difference between array placement and row iteration, and how both interact with card quantity in column A.

5.3.1 Why this matters

At first sight, these two expressions look similar:

```
[id1 id2 ... idn]
*[id1 id2 ... idn]
```

They are not equivalent. The first builds one composite placement inside one generated card. The second expands the dataset row into multiple generated cards, one per iterator value.

5.3.2 Array Without Iterator ([...])

When a non-text field uses:

```
[id1 id2 ... idn]
```

PnPink treats it as an array/group target for that field in the current card instance. All listed items are processed in the same row instance.

This is useful when you want several IDs resolved together in one card, and optionally apply local array layout/fit behavior.

Wildcard IDs are supported inside arrays:

```
[main_icon-*
```

This expands to all matching IDs and keeps array/list semantics (single card instance).

Array repetition shorthand is also supported:

```
[3*:Ic(potato)]~i2
```

Equivalent to writing the same token three times in the list.

Grouped repetition is also supported with parentheses:

```
[id1 - 3*(id2 3- 3*id3) 4- id4]
```

Parentheses are grouping operators inside list/array bodies, so you can repeat a block as one unit.

5.3.3 Iterator (*. ...)

When a cell starts with `*`, it becomes an iterator expression.

```
*[id1 id2 ... idn]
```

Result:

- the row is expanded into multiple internal instances,
- each generated card gets one iterator value at that cell,
- by default (without explicit copies), generated cards count equals iterator length.

Conceptually: one different card per value.

Parentheses inside iterator lists

Inside `*[ ... ]`, parentheses define one grouped iterator item (multivalue in the same generated card instance):

```
*[id1 id2 id3]
```

This iterates 3 scalar items (`id1`, then `id2`, then `id3`).

```
*[id1 (id2 id3)]
```

This iterates 2 items:

- first iteration: `id1`
- second iteration: `id2 id3` (multivalue group in the same card)

So `*[id1 id2 id3]` and `*[id1 (id2 id3)]` are intentionally different.

Group repetition also works:

```
*[id1 2*(id2 id3)]
```

Equivalent iterator sequence: `id1`, `(id2 id3)`, `(id2 id3)`.

Wildcard IDs are also supported in scalar/multivalue cells:

```
main_icon-*
```

Outside `[ ... ]`, wildcard expansion behaves as multivalue (same card instance, token sequence order).

5.3.4 Multi-Level Iteration (*, **, ***)

PnPInk supports multiple iterator levels using leading stars.

```
*[A B C]
**[1 2]
```

This produces a nested expansion (cartesian-style): every level-1 value is combined with every level-2 value. You can add more levels (`***`, etc.) when needed.

5.3.5 Supported Iterator Expressions

Current parser supports iterator payloads such as:

- bracket list/range: `*[1 2 3]`, `*[1..10]`, `*[A..F]`,
- virtual sources with selectors,
- filesystem glob through `*@{...}`,
- spritesheet wildcard alias: `*@alias[*]`.

If an iterator resolves to zero values, that row yields zero generated instances.

5.3.6 Copies in Column A (Card Quantity)

Column A can declare explicit quantity (copies) for a row. With iterators, behavior is:

1. No explicit copies in column A: Iterator count defines card count.
2. Explicit copies in column A and copies > iterator count: Iterator values wrap around.
3. Explicit copies in column A and copies < iterator count: Iterator sequence is truncated.

This gives predictable control when you need either:

- one card per iterator value,
- or a fixed number of cards regardless of iterator length.

5.3.7 Examples

One card with grouped IDs

```
art = [icon_fire icon_air icon_earth]
```

Interpretation: one row instance, one generated card, grouped placement in that card.

One card per ID

```
art = *[icon_fire icon_air icon_earth]
```

Interpretation: three generated cards from that row (unless column A copies overrides).

Iterator + explicit copies

```
Column A: 5  
art = *[A B C]
```

Interpretation: 5 cards -> A, B, C, A, B (wrap).

```
Column A: 2  
art = *[A B C]
```

Interpretation: 2 cards -> A, B (truncate).

5.4 Fit-Anchor

Fit-Anchor is one of the most important modules in PnPink. It is the placement engine that lets you position objects relative to placeholders without manual coordinates and without hardcoding object/template sizes.

In practice, Fit-Anchor lets you place one or many objects on a placeholder and then align, scale, rotate, mirror, shift, and clip them with compact syntax.

5.4.1 Dataset Mental Model

Fit-Anchor is easier to understand if you keep this model in mind:

- the dataset header points to a placeholder object in the SVG,
- each row cell provides one or many source objects,
- Fit-Anchor defines how those source objects are positioned relative to that placeholder for each generated card/instance.

5.4.2 What Is an "Object" Here?

In this documentation, we call IDs "objects" (not "elements") from the DSL/user point of view.

An object can be:

- an existing SVG object by ID (rect, path, group, text, image, etc.),
- a dynamic object created by [Source](#) (local file, spritesheet frame, internet URL, Wikimedia/Pixabay, icon library),
- a multivalue object list in one cell: `id1~7 id2~9 id3~3`,
- an explicit object array: `[id1 id2 id3]` (optionally with local array layout).

Representative examples:

```
main_art-8
heart_icon
@{assets/picture.png}~i5
@sp1[14]~m5
@{wkmc://File:Example.svg/svg#nodeX}~i5
id1~7 id2~9 id3~3
[id_a id_b id_c].L{3x1 g=2}~i5
```

5.4.3 Syntax and Style

Fit-Anchor follows the same DSL conventions as other modules:

- `Module{ key=value ... }` long form,
- short aliases (`a`, `b`, `s`, etc.),
- default parameters can be omitted,
- compact shorthand with `~` is available and heavily used in real datasets.

Long form:

```
object_id.Fit{ border fitmode anchor shift clip rotate mirror }
```

Compact form:

```
object_id~...
```

Compact Example (Explained)

Example:

```
id1~[10%]i7^^[-50% 0]!
```

Read it left to right:

- `[10%]` -> border (resize context around placeholder),
- `i` -> fit mode `inside`,
- `7` -> anchor top-left,
- `^^` -> rotate 180 deg,
- `[-50% 0]` -> shift left by half placeholder width,
- `!` -> clip to the placeholder shape.

This compact style is very expressive once you know the token order.

5.4.4 Fit-Anchor Parameters

The following subsections describe each Fit-Anchor parameter and what it changes in final placement. Order matters conceptually.

Border (`b=`)

Border changes the usable fit area around the placeholder.

- positive values expand,
- negative values shrink.

Important behavior:

- `border` modifies placeholder size **only for Fit Mode scaling**.
- `anchor`, `shift`, and `clip` still use the **original placeholder** as geometric reference.

In other words:

1. border adjusts the fit area,
2. fit mode computes object scale in that adjusted area,
3. anchor/shift/clip are evaluated against the original placeholder.

This is why defining `border` first is conceptually important: it controls object size before final positioning.

Examples:

```
border=[-2]
border=[2 3]
border=[2 3 4 5]
object_id~[-2]i
```

Percent and absolute-size forms are supported:

```
border=[50%]
border=[125%]
border=[40x60]
border=[50%x20]
border=[23x?]
border=[?x12]
```

Negative `WxH` components can flip orientation:

```
border=[100%x-100%]
border=[-100%x100%]
```

Fit Mode

Fit mode defines scaling behavior relative to the (possibly border-adjusted) fit area. If omitted, `inside` is the practical default.

Examples:

```
object_id.Fit{fitmode=inside}
object_id~i
```

Fit Mode Reference

Code	Name	Description
i	inside / contain	Scales proportionally to fit entirely within the rect (default).
o	original / none	Keeps original size.
w	width-fit	Scales proportionally to match rect width.
h	height-fit	Scales proportionally to match rect height.
m	max / cover	Scales proportionally until it covers the rect, possibly overflowing.
x	x-stretch	Stretches width to match rect (non-proportional).
y	y-stretch	Stretches height to match rect (non-proportional).
a	all-stretch	Scales independently in X/Y to fill rect exactly.
t	tile	Tiles the object as a pattern within the rect.
b	best-fit	Smart mode that mixes m, a, and clipping for balance.
?	auto-fit	Alias of <code>b</code> (best-fit).

Anchor

Anchor selects the reference point used to align an object to the placeholder. Think of it as: "which point of the object goes to which point of the placeholder."

Anchor uses the **original placeholder** (not the border-adjusted fit area). This makes anchor behavior stable and predictable while `border` only affects scale.

`anchor = 1..9` follows numeric keypad:

7 8 9

4 5 6

1 2 3

Examples:

```
object_id.Fit{anchor=7}
object_id~7
```

If no explicit anchor is provided, center (`5`) is used as default.

Shift (`shift=` / `s=`)

Shift offsets final position after anchor/fit.

```
object_id.Fit{anchor=7 shift=[-100% -100%]}
object_id~i8[0 %]
object_id~i8[-50% 0]
```

Notes:

- `%` is relative to placeholder size,
- `%` means `100%`,
- `-%` means `-100%`,
- mixed expressions are valid (`shift=[25%+2 0]`).
- shift uses the original placeholder frame as reference.

Rotate (`r=` / `^`)

Rotates the target object.

```
object_id.Fit{rotate=-42.4}
object_id~^^^
object_id~^~4517
```

Mirror (`m=` / `|`)

Mirrors the target.

- `|` horizontal,
- `||` vertical.

```
object_id.Fit{mirror=v}
object_id~||
```

Clip (`c` / `!`)

Clips outside the original placeholder shape.

```
object_id~i8![0 %] # pre-clip then shift
object_id~i8[0 %]! # shift then post-clip
```

Position of `!` matters.

5.4.5 Compact Token Order (Reference)

When using shorthand `~`, tokens are parsed in this order:

1. Optional border list (`[t r b l]`, `[x]`, `[x y]`)
2. Fit mode + anchor (`i7`, `m5`, `a9`, etc.)
3. Optional shift list (`[dx dy]`)
4. Optional clip (`!`)
5. Optional rotation (`^deg`, `^^`, `^^^`)
6. Optional mirror (`|`, `||`)

5.4.6 Priority and Overrides

When multiple Fit-Anchor layers apply, use this precedence:

1. Header defaults/global ops
2. Iterator/global ops
3. Item-local ops

Later layers override earlier ones for conflicting properties.

Example:

```
Header:
main_art-8=~[10x10]

Cell:
*[:fig(g918)~^^ :fig(g1720) :fig(g1264) :fig(g13590)]~[0%]5
```

Effective merge:

- start from header border,
- apply iterator-level defaults,
- apply item-level overrides last.

5.4.7 Practical Examples

This section intentionally groups compact real-life patterns.

5.4.8 Compact Notation (Clarifications)

This section consolidates the compact syntax details that usually cause confusion.

With ~ and without ~

For simple rotate/mirror/clip operations, ~ can be omitted:

- `id~^15` and `id^15` are equivalent.
- `id~!` and `id!` are equivalent.
- `id~|` and `id|` are equivalent.
- `id~||` and `id||` are equivalent.

Use the ~ form when you want the full compact chain in one expression (`border + fit/anchor + shift + clip + rotate + mirror`).

Combined compact operations

You can combine these operators in one token, for example:

```
id~i5^^!
id^90|
id~m7[0 5%]!
```

Parsing and precedence reminder

- The compact parser still applies the same Fit-Anchor merge precedence: header defaults -> iterator/global ops -> item-local ops.
- For conflicting properties, the last layer overrides previous ones.

Example 1: Simple inside-center

```
art_id~i5
```

Places object inside placeholder, centered.

Example 2: Cover and clip

```
art_id~m5!
```

Covers placeholder area, then clips overflow to placeholder shape.

Example 3: Corner icon with offset

```
icon_id~i7[2 -2]
```

Inside fit, top-left anchor, then fine offset.

Example 4: Multiple objects in one cell

```
id1~7 id2~9 id3~3
```

Places three objects in one placeholder flow, each with its own anchor.

Example 5: Array group with local layout

```
[id1 id2 id3].L{3x1 g=2}~i5
```

Builds an array object, lays it out locally, then applies Fit-Anchor as one grouped target.

5.5 Layout

Defines how a group of elements are arranged in a grid (e.g. cards on pages). Use this module when you want deterministic pagination and slot planning.

5.5.1 Syntax

```
Layout{ p=nxm g=[x y] o=[w1 h1 w2 h2] shape=... } or L{nxm g=... o=... s=...}
```

Can be applied to any object:

```
rect.L{}
```

```
[rect1 rect2 ...].L{}
```

```
{A4}.L{p=5x8 s=hexgrid}
```

5.5.2 Pattern (p=)

`p` is the structural core of layout. It defines how many slots exist per page before page breaks occur.

`p=` (or positional token) defines columns x rows.

```
p=3x2
p=0x0
p=3x?
p=?x4
p=-?x-?
```

`0x0` means auto-fit based on available area. `?` is also accepted as auto, so `0` and `?` are equivalent in pattern.

Order and Flips

Order and flips matter when instance numbering and print order must follow a specific physical workflow.

The grid token supports modifiers:

- `^` switches order to top-to-bottom, then left-to-right.
- `|` flips the axis (use after the number you want to flip).
- Negative numbers also flip that axis.

Examples:

```
p=3x2      -> left-to-right, top-to-bottom
p=3x2^     -> top-to-bottom first
p=3|x2     -> flip columns
p=3x-2     -> flip rows
p=-3x-2    -> flip both axes
```

5.5.3 Gaps (g=)

Use gaps to control spacing between slots without changing card/template size.

`g=` (or `gaps=`) defines spacing between slots.

```
g=[x y]
g=2
g=?
```

Rules:

- 1 value means `x=y`.
- 2 values mean `x` (horizontal) and `y` (vertical).
- Units and percentages are allowed.
- `?` means auto gap (distribute remaining space evenly to fit the final content area on that axis).

Examples:

```
g=2
g=[2 3]
g=[1% 3%]
g=[2 ?]
g=[? ?] # same as g=[?] and g=?
```

Percentages require a known card size (shape preset or template size).

5.5.4 Offset (o=)

Offsets are useful for staggered layouts, including hex-like placement. They modify slot position patterns, not slot dimensions.

`o=` (or `offset=`) defines staggered offsets for alternating slots.

```
o=[w1 h1 w2 h2]
```

Notes:

- If only `w1 h1` are provided, `w2 h2` defaults to `-w1 -h1`.
- Offsets are applied after slot sizing and before final placement.

This is how hex-like or staggered grids are achieved without changing card size.

5.5.5 Shape (s=)

Shape presets provide normalized card/tile sizes so layouts remain consistent across projects.

Defines the shape preset of each grid cell.

Examples:

```
s=poker
s=minieuro
s=55.3x77.1
s=rect<55.3x77.1>
s=hex<24x33>
s=polygon<[5 23x32]>
```

See the Presets page for the full list of named sizes.

Hex Shapes

Hex shapes are specialized layout modes for maps and tile production. They automate spacing behavior that would otherwise require manual offset tuning.

Smart shapes adjust gaps and offsets without changing card size:

- `s=hexgrid` for maps, boards, overlays.
- `s=hextiles` for cuttable tiles with shared edges.

Inkscape tip: use **Tools > Stars and Polygons** (Shift+F9), set corners to 6, and hold Ctrl while resizing to keep alignment.

5.6 Page

Defines page format, margins, and the global page cursor. Use `Page{}` whenever print format, orientation, or pagination position must change.

5.6.1 Syntax

```
Page{ size landscape border at } or P{A4} or {A4}
```

When `Page{}` appears in the first column, the keyword `Page/P` can be omitted. A bare `{}` means "continue using the previous page size".

Multipliers are allowed:

```
{3*A4} -> three A4 pages
{3}    -> shorthand for three pages
```

5.6.2 Size (default)

Page size is usually set once and then reused by state.

The default parameter is the page size:

```
Page{A4}
Page{Letter}
Page{23.3x34.45}
```

5.6.3 Landscape / Portrait

Orientation is part of page state and affects all following placements until changed.

```
Page{A4^}    -> landscape
Page{landscape}
Page{portrait}
```

5.6.4 Border (b=)

`b` defines the usable inner area (or outward expansion) for layout planning.

`b=` defines padding or margin around the page.

```
b=[-2]    -> 2 mm inward margin
b=[2 3 4 5] -> top, right, bottom, left
```

Percentages and absolute `WxH` values are allowed (same grammar as `Fit`).

For `%` borders in 1/2-token forms, percentages define absolute target scale:

- `b=[50%]` -> final size x0.5
- `b=[125%]` -> final size x1.25
- `b=[50% %]` -> height x0.5, width x1.0
- `b=[23x?]` / `b=[?x12]` are valid in `WxH` absolute-size mode (`?` = 100% on that axis).

5.6.5 Page Cursor (at / a / @)

Cursor control is for advanced pagination workflows, such as merging sections or forcing output positions.

`at` moves the global page cursor.

Accepted forms:

```
at=+3  
a=-1  
@5  
A4@+2  
{ @-1 }
```

Rules:

- `+n` / `-n` -> relative move.
- `n` -> absolute 1-based page (n -> index n-1).
- If the current page already has content, the engine jumps to a new page first.

5.7 Source

`@{...}` creates a source object from an external resource (file, icon, or URL), then uses it as a renderable object in the document.

Use Source when content comes from outside the template and must be resolved at generation time.

5.7.1 Syntax

All three forms below are equivalent entry points into the same source resolver pipeline.

```
Source{source_ref} or S{source_ref} or @{source_ref}
```

The `source_ref` can be:

- a path to a local file,
- an iconify reference,
- a web URL,
- or a virtual source (Wikimedia Commons, Pixabay, Openclipart).

5.7.2 Local File Sources

Use local files for stable, reproducible builds where assets are versioned with the project.

```
@{ relative/or/absolute/path.png }
@{ C:\path\to\image.png } # Windows
@{ ~/images/token.png } # Linux/macOS
```

Equivalent local-source forms accepted in dataset cells:

```
Source{path/file.ext}
@{path/file.ext}
@{file.ext}
@file.ext
file.ext
```

Notes:

- `file.ext` is accepted only for known source extensions (`png`, `jpg`, `jpeg`, `gif`, `bmp`, `webp`, `svg`, `svgz`, `pdf`, `tif`, `tiff`).
- `file.ext}` is **not** valid syntax. The closing `}` only exists in braced forms:
- `@{file.ext}`
- `Source{file.ext}`

When no path is provided (`file.ext`), lookup order is:

1. Same folder as the SVG.
2. `assets/`
3. `images/`
4. `img/`
5. `imgs/` (compatibility)

Local sources support environment/home expansion:

- Windows: `%USERPROFILE%`
- Linux/macOS: `$HOME`, `${HOME}`, `~`

Inline text tokens

Inline icons can also use local shorthand directly inside text:

```
:bola.png:
```

Behavior:

- If the file is found by the same lookup rules above, it is treated as a source token.
- If not found, the token is kept as normal literal text (`:bola.png:`).

5.7.3 SVG Node Import (Optional)

For SVG sources only, you can target a specific node with `#id`:

```
@{ assets/icons.svg#heart_group }
@{ https://example.com/icons.svg#token_1 }
```

Behavior:

- without `#id`: the whole SVG is linked as a final image source,
- with `#id`: that node is imported and embedded (including required defs/references).

5.7.4 Iconify Sources

Use icon sources for semantic symbols (costs, resources, status icons) without managing local files manually.

```
@{ icon://icon_set/icon_name }
```

Examples:

```
@{ icon://noto/heart-suit }
@{ icon://mdi/account }
@{ icon://cat } # uses default set (noto)
```

There is a default snippet definition mapping to the `noto` icon set:

```
:Ic(icon) -> @{ icon://noto/icon }
```

Notes:

- Icons are cached as SVG symbols in `<defs>`.
- If `iconify.py` is not available, a placeholder is created.
- To force a re-download, remove the symbol from `Object > Symbols` (Shift+Ctrl+Y).

5.7.5 Web Sources (HTTP/HTTPS)

Use web sources when assets are remote and can be cached at generation time.

```
@{ https://... }
@{ http://... }
```

Web sources are **downloaded and cached** into the `assets` folder next to the SVG (or project root). If the download fails, a placeholder symbol is created.

5.7.6 Virtual Sources

Virtual sources map high-level search expressions to real downloadable URLs. They are useful for exploratory workflows and rapid prototyping.

PnPink supports virtual sources that resolve to real URLs:

```
@{ wkmc://query/size }
@{ pxby://query/size }
@{ oclp://query/size }
```

- `wkmc://` (Wikimedia Commons) supports:
 - search text: `wkmc://query/size`
 - specific file: `wkmc://File:Name.ext/size`
 - category files: `wkmc://Category:Name/size`
- `pxby://` (Pixabay) supports:
 - normal search list: `pxby://query/size`
 - direct lookup for no-space query when possible (single hit / id).
- `oclp://` (Openclipart) supports:
 - normal search list: `oclp://query/size`
 - specific image by id/detail: `oclp://id:12345/size` (resolved internally via <https://openclipart.org/detail/12345/...>)

Size accepts:

- presets: `tiny`, `small`, `medium`, `large`, `xlarge`, `largest`
- vector mode: `svg` (when available from the provider)
- minimum side: `N` (example: `1000`)
- minimum width+height: `WxH` (example: `1000x1000`)

Examples:

```
@{ wkmc://"The Ancestral Homes of Britain"/large }
@{ wkmc://"File:Complete_Saxonian_deck.jpg"/1000 }
@{ wkmc://"Category:Complete_decks_of_playing_cards_laid_out"/1000x1000 }
@{ pxby://castle/1200 }
@{ oclp://wolf/large }
@{ oclp://id:24829/largest }
```

Multiple-result virtual sources can be selected outside the source with a 1-based selector:

```
@{ wkmc://"The Ancestral Homes of Britain"/medium }[2 4..12 15..26]
*@[ pxby://castle/large }[1..20]
```

Selector notes:

- indices are 1-based,
- out-of-range indices are ignored with warning,
- without selector, if multiple results exist, the first one is used (warning in log).

5.7.7 Fit and Placement Behavior

After resolution, a source behaves like any other placeable target in Fit/Anchor terms.

A Source is treated as a final placeable object, so you can apply Fit/Anchor operations exactly as with other objects.

5.7.8 Internal Model

This internal model is important for performance and file size on large projects.

Sources are instantiated as clones of symbols (`<use>`), not inline geometry copies. This keeps the SVG compact and avoids repeating heavy geometry.

5.7.9 Spritesheets (@)

Spritesheets allow one asset to provide many addressable frames. This is useful for token sheets, icon atlases, and catalog-like image packs.

The **Layout** module can also be applied to composite sources, such as spritesheets.

Example definition inside a comment block (before the dataset):

```
# @sp1 = @{sheet.png}.Layout{p=3x2^ s=poker g=[4 3]}
```

This creates a spritesheet from `sheet.png` with a 3x2 grid of poker-sized cards, spaced 4 mm horizontally and 3 mm vertically.

Then, you can reference any frame in the dataset as:

```
@sp1[14] -> frame 14 (third page, second row)
@sp1[2][1][3] -> page 2, column 1, row 3
@sp1[1..6] -> range selector
@sp1[1 4 7] -> explicit list selector
```

(`^` in the grid reverses numbering direction: first rows, then columns.)

This spritesheet can be used as a source in any dataset, positioned with Fit parameters (e.g. `~i6`) as any other target.

Notes:

- Alias definitions are read from comment lines before dataset rows.
- Frame selectors are 1-based.
- If a frame is out of range, a warning is logged and placement is skipped.

5.8 Marks

The Marks module generates cut marks aligned to the **Layout grid slots** of the **main template** (first dataset column). Use it when output is intended for physical cutting and alignment consistency matters.

Marks are defined in the **first header cell** (same place as `Page{}` and `Layout{}`), for example:

```
{A4}.L{p=3x3 g=2}.M{ mk_style len=[3 2] d=2 }
```

5.8.1 Slot-Based Behavior

This behavior explains why marks remain correct even when pages, gaps, or offsets change.

Marks are generated **per slot** (per placed instance in the grid), using the transformed bounding box of the main template for that slot.

This means:

- Marks follow the real grid placement (including gaps, offsets, page breaks, rotations, etc.).
- Marks are not tied to a specific template column; they follow the main layout slots.

5.8.2 Syntax

The syntax is short, but each parameter controls a different geometric aspect of cut lines.

```
Marks{ style len distance border layer }
```

Shorthand:

```
M{ ... }
```

Elements inside `{ }` are space-separated. Lists are written in `[ ]` and are space-separated.

5.8.3 Default Parameter (Style)

Style controls visual appearance, not mark geometry. It is resolved from existing SVG elements.

The default parameter of `Marks{ }` is the style id. This means `style=` can be omitted:

```
.M{ mk_style len=3 d=2 }
```

is equivalent to:

```
.M{ s=mk_style len=3 d=2 }
```

Style Sources

A style id (`s=`) references an SVG element by ID:

- If the ID refers to a single element (path/shape), its stroke-related style is copied to the generated mark paths.
- If the ID refers to a group (`<g>`) containing multiple child paths, it is treated as a **style stack**. The mark geometry is generated multiple times, each copy receives the style of one child, and the result is layered.

If no `s` is provided, a default style from preferences is used.

5.8.4 Length (len=)

Length controls how far marks extend outside and inside slot edges.

`len=` defines the length of the mark segments.

Rules:

- Scalar: `len=3` means **external length = 3**, internal length = 0.
- Two-value list: `len=[out in]`.

If the layout uses offsets (staggered grids), a scalar `len=3` is treated as `[3 3]` to keep internal marks visible.

5.8.5 Distance to Card (d=)

Distance is the gap between the card edge and the start of the external mark segment.

`d=` sets the distance between the card edge and the **external** mark origin.

Default: `d=2`.

`d` follows the same list grammar as border in Fit/Page:

```
d=[2 3 4 5] -> top, right, bottom, left
```

5.8.6 Border Pattern (b=)

Border offsets let you fine-tune mark placement against bleed/margin strategies.

`b=` defines the cut-mark border offsets using the same conventions as borders in Fit/Page.

Default: `b=0`.

5.8.7 Output Layer (layer=)

Separating marks into a dedicated layer helps review, export, and printer handoff.

`layer=` defines the target Inkscape layer where marks will be inserted.

Default: `layer=marks`.

If the layer does not exist, it is created.

Example:

```
.M{ layer=cutmarks }
```

5.8.8 Scope

This section documents current implementation limits to avoid false expectations.

The `scope=` parameter is documented but **not implemented** in the current engine. Marks are always generated on the front pass.

5.8.9 Examples

Basic cut marks with default style

```
{A4}.L{p=3x3 g=2}.M{ }
```

Using a style element

```
{A4}.L{p=3x3}.M{ mk_cut }
```

Overlay style using a group

```
{A4}.L{p=3x3}.M{ mk_cut_stack len=[3 2] }
```

5.9 Gradients (Experimental)



Warning

This feature is currently **in development**. Syntax and behavior can still change in future versions.

5.9.1 Overview

PnPInk can create SVG `linearGradient` definitions directly from dataset comment directives. These gradients are generated in `<defs>` and can then be used from any SVG style, for example:

```
fill: url(#gradientX);
stroke: url(#gradientX);
```

5.9.2 Where to define gradients

Gradient definitions are read from dataset comment lines (same global/local comment mechanism used by snippets and spritesheets).

Typical location:

- top comment block of your dataset file,
- before data rows,
- usually with `#` at the start of the first cell.

Example:

```
# gradientX=G{b88326ff^25 [-25 12 5] [#BA8726FF 35 20 10]}
```

5.9.3 How to apply gradients

Once a gradient ID exists, apply it in your SVG element style:

```
style="fill:url(#gradientX);"
```

Or as stroke:

```
style="stroke:url(#gradientX);"
```

PnPInk does not require a special dataset field for this part. You use normal SVG styling with the generated gradient ID.

5.9.4 Syntax

Two forms are supported.

Short form (`G{...}`)

```
# gradientX=G{basecolor^angle [stop1] [stop2] ...}
```

Example:

```
# gradientX=G{b88326ff^25 [-25 12 5] [#BA8726FF 35 20 10]}
```

Meaning:

- `basecolor`: base color of the gradient (`#RRGGBB` or `#RRGGBBAA` , `#` optional).
- `^angle`: linear gradient angle in degrees (optional, default `0`).
- each stop block: `[...]` definition for local highlight/shadow behavior.

Long form (`Gradient{...}`)

```
# gradientX=Gradient{basecolor=b88326ff rotate=25 stops=[[-25 12 5] [55 35 20 10]]}
```

Parameters:

- `basecolor` (or `base / color`): required.
- `rotate` (or `angle / r`): optional, default `0` .
- `stops=[...]`: required in long form.

5.9.5 Stop definition

Each stop can be numeric-light or explicit-color.

Numeric light stop

```
[light pos a b]
```

- `light`: `-100..100`
- positive -> lighter (towards white),
- negative -> darker (towards black).
- `pos`: position in `%`.
- `a`: left width in `%`.
- `b`: right width in `%` (optional, if missing then `b=a`).

Example:

```
[-25 12 5]
```

This creates a darker zone centered at `12%`, with left/right width `5%`.

Explicit color stop

```
[color pos a b]
```

- `color`: `#RRGGBB` or `#RRGGBBAA` .
- `pos`, `a`, `b`: same meaning as above.

Example:

```
[#BA8726FF 35 20 10]
```

This creates a custom-color peak at `35%`, with asymmetric widths (`20%` left, `10%` right).

5.9.6 Behavioral details

Current implementation rules:

1. Base color is present at `0%` and `100%`.
2. Each stop adds three internal points:
3. `(pos-a) -> base color`
4. `pos -> peak color`
5. `(pos+b) -> base color`
6. Offsets are clamped to `0..100%`.
7. If multiple points collapse to the same offset, last generated value wins.
8. `light` values are converted by mixing base color with white/black.

5.9.7 End-to-end example

Dataset comments:

```
# gradientGold=G{b88326ff^25 [-25 12 5] [55 35 20 10]}
```

SVG object:

```
<rect id="card_bg" style="fill:url(#gradientGold);stroke:#000;stroke-width:0.3" />
```

Result:

- `gradientGold` is created in `<defs>`,
- object fill resolves to the generated linear gradient.

5.9.8 Current limitations

- Only `linearGradient` is generated.
- `radialGradient` and mesh gradients are not part of this feature yet.
- Because this is experimental, parser details and interpolation behavior can evolve.

5.10 Advanced

Advanced DSL features that are implemented in code and useful in larger projects.

5.10.1 Alias Definitions

Aliases reduce repetition in large datasets and make rows easier to read.

You can define aliases and reuse them later:

```
@hero = @{{assets/hero.png}}
@icons = [icon_hp icon_atk icon_def]
```

Then reference by index:

```
@hero
@icons[2]
@icons[1..3]
@icons[*]
@icons[1 4 7]
```

5.10.2 Source Suffixes

Suffixes are useful when source resolution and fit behavior must be expressed in one token.

Source expressions support both long Fit and shorthand ops:

```
@{{assets/token.svg}}.Fit{{mode=i anchor=5}}
@{{assets/token.svg}}-i5^15|
```

This applies the same Fit/Anchor logic used for normal SVG IDs.

5.10.3 Page Cursor Control (`at`)

Use cursor control when output must start at a specific page position.

`Page{}` supports page cursor movement with `at` / `a` / `@`:

```
{A4 @+3}
Page{A4 at=-1}
Page{A4^@5}
```

This controls where the next generated content starts in the global page sequence.

5.10.4 Page Break Blocks

These forms are useful for explicit pagination control between logical sections.

Standalone page blocks are valid and useful:

```
{ }      # break to next page
{3}      # advance by 3 pages
{3*A4}   # multiplier + size
```

5.10.5 Leading Cell Composition

Combining directives in column A allows row-level orchestration without extra columns.

Column A data rows can combine multiple directives in one cell:

```
{A4 b=[-5]} .L{p=3x3 g=2} .M{{mk_default d=2}} [10 3- 5]
```

Supported order is flexible, but the parser expects:

1. Optional `Page{}` block.
2. Optional `.M{}` marks block.
3. Optional `.L{}` layout block.
4. Optional copy/hole tail (`[10 3- 5]` or trailing number).

5.10.6 Inline Icon Tokens in Text

These tokens allow icon insertion directly in flowing text while preserving typographic behavior.

Text rendering supports inline icon tokens:

```
:heart_icon:
:@{icon://noto/heart-suit}:
:@{icon://noto/star}~i5^10:
```

Token forms implemented:

- `:id:` uses an existing SVG id.
- `:@{...}:` uses a Source token with optional fit suffix.
- `:S{...}:` and `:Source{...}:` are equivalent source forms.

6 PnPInk User Manual

6.1 Introduction

PnPInk is a **free, open-source, and cross-platform toolkit** designed to create high-quality *print-and-play* materials for board games -- cards, boards, tokens, player aids, and more -- directly inside **Inkscape**.

Unlike other generators, **PnPInk is not a separate application but a native extension suite that transforms Inkscape into a complete publishing environment -- capable of accompanying creators from the very first prototype designs to the final, fully professional print composition.**

6.2 What PnPInk Does

PnPInk automates the creation of:

- **Card decks** of any size or format.
- **Boards and punchboards**, including **hexagonal or square grids**.
- **Player sheets**, tiles, counters, and rule inserts.

It combines **data-driven generation** (from Google Sheets or CSV files) with **Inkscape's full graphic power** -- gradients, filters, clipping masks, paths, symbols, and layers -- to produce editable, print-ready layouts with pixel-perfect precision.

6.3 Key Principles

Principle	Description
Open Source	PnPInk is released under an open license. All files remain accessible, editable, and transparent.
Cross-Platform	Works wherever Inkscape runs: Windows, macOS, or Linux.
Non-Destructive	Everything you generate stays as pure SVG -- fully editable afterwards.
Professional Output	Exports PDFs with CMYK color , ICC profiles, and print standards (FOGRA, PDF/X).
Accessible & Visual	No programming required -- all actions are done through menus, dialogs, and clear visual feedback.

6.4 Power of Inkscape Integration

Because PnPInk is built *inside* Inkscape, it inherits all of its advanced features:

- **Full vector control:** Bezier paths, shapes, and text on path.
- **Complex styling:** gradients, blurs, blend modes, and filters.
- **Image management:** linked or embedded images, with live previews.
- **Multi-page support:** each layout can produce complete, paginated projects.
- **Universal format compatibility:** SVG, PDF, PNG, EPS, DXF, and more.

This makes PnPInk suitable for both quick prototypes and professional offset printing.

6.5 Intelligent Data and Automation

PnPInk can read structured data directly from:

- **Google Sheets** (live connection with authentication), or
- **CSV files** exported from any spreadsheet program.

Each row in the data corresponds to a card, tile, or component.\ PnPInk replaces variables automatically -- names, values, icons, or images -- to generate all items in a single command.

You can also control elements through short, **intuitive syntax rules**:

- define page size and margins (`{A4 b=[5 4]}`),
- define the grid (`.L{p=3x3 g=2}`),
- position elements with anchors (`ID~i8` or `ID.Fit{a=8}`).

This nomenclature is **compact, logical, and extremely powerful**, allowing advanced layouts without complex scripting.

6.6 Extensible by Design

PnPInk includes an **internal Python engine** that lets power users create custom behaviors, filters, or generation logic.\ At the same time, the system remains compatible with **external programming** through Google Sheets formulas, scripts, and data manipulation -- perfect for dynamic prototypes or large datasets.

Together, these layers make PnPInk both **simple for beginners** and **limitless for experts**.

6.7 Output Quality and Professional Printing

All final projects can be exported as:

- Editable SVG files for on-screen review.
- **Print-ready PDFs** with CMYK color conversion and **bleeds, margins, and crop marks**.
- Optional **PDF/X compliance** for professional pre-press workflows.

This ensures consistent color reproduction and compatibility with professional printers and services.

6.8 Typical Use Cases

- Design and print your own card game in minutes.
- Generate hundreds of cards from a single Google Sheet.
- Build modular or hexagonal boards with automatic alignment.
- Create color-managed PDFs for offset printing.
- Share open, editable project files with collaborators worldwide.

6.9 Why PnPInk

PnPInk bridges the gap between **creativity** and **production**.\ It combines the artistic freedom of Inkscape with the precision and automation of modern data tools -- delivering a unique environment where *design, logic, and craft meet*.

6.10 PnPInk Basic Workflow (User Guide)

6.10.1 What you need

- **Inkscape** (Windows, macOS, or Linux).
- **PnPInk extension** installed. After installing, **new menus** appear in Inkscape under **Extensions > PnPInk DEV** (e.g. **DeckMaker v0.24dev**).

6.10.2 Your first project: the quick overview

1. **Design the template** in Inkscape\ Create a single "master" card (or board). This can be a full finished design, or a **generic frame** with placeholders (title, number, icon, art frame, background, etc.). This will be your **TEMPLATE**.
2. **Name things clearly** Give each element a meaningful ID (e.g., title, cost, icon, art, bg). IDs let PnPInk know** what to replace** later.
3. **Prepare your data** Use Google Sheets or a CSV file. Each row represents one card (and can include "copies" or quantity). Add columns that match your IDs--plus any other properties you want to control (texts, colors, styles, images, counts, etc.). There's no limit**** to how many you define.
4. **Choose a layout** Tell PnPInk how to arrange cards on pages (paper size, grid, margins, gaps). Layouts are written with a short, intuitive notation**** (e.g., `{A4 b=[5 4]}` with `.L{p=3x3 g=2}`).
5. **Generate** PnPInk clones the template for each row, replaces variables, positions/adjusts elements with Anchor & Fit, arranges them into multi-page documents, and prepares print-ready PDFs (including CMYK/ICC**** for professional printing).

6.10.3 IDs: how elements are matched to data

- **What they are:** IDs are the "names" of objects in your template (texts, shapes, groups, images).
- **How to use them:** If your CSV/Sheet has a column **title**, and your template has a text object with ID **title**, PnPInk will insert that row's text there.
- **Best practices:**
 - Keep IDs **short and descriptive** (title, subtitle, icon, art, frame, bg).
 - One purpose per ID (avoid duplicates unless you intend to repeat the same value).
 - Use **groups** with an ID when multiple items belong together (e.g., a framed art block).

6.10.4 Layouts & pagination (short notation)

- **What layouts do:** They control paper size, card size/quantity per page, margins, and gaps.
- **Notation examples:**
 - `{A4}` with `.L{p=3x3}` -> A4 paper, 3 columns x 3 rows.
 - `{A4 b=[5 4]}` with `.L{p=3x3 g=2}` -> margins and gaps.
- Orientation toggles and presets let you switch between portrait/landscape and standard card formats.
- **Why it matters:** One template can produce **hundreds of cards**, arranged automatically into **clean, paginated sheets**--ready for cutting or die-cut.

6.10.5 Anchor & Fit (precise placement without manual nudging)

- **Anchor** = where the element aligns inside its target area (e.g., 8 = top-center, 5 = center, 3 = bottom-right).
- **Fit** = how it scales to **fill or respect** the frame:
- none (no scaling),

- contain (keep ratio, fit inside),
- cover (keep ratio, fill completely, may crop),
- stretchXY (scale independently).
- **Margins and bleed:** You can add **margins or negative margins** (e.g., [2] = 2mm padding on all sides; [-1] = extend 1mm beyond for bleed).
- **Result:** Titles center perfectly, icons snap to corners, images fill art frames, and backgrounds expand to edges--**consistently** across every card.

6.10.6 Inline icons (type the name, get the icon)

- **What it is:** Write icon names directly in text (e.g., :sword:, :lightning:) and PnPInk replaces them with **vector icons**--styled to match your fonts and colors.
- **Scale & style:** Icons live **inside the text flow**, so they line up with your typography and inherit styles.
- **Big libraries:** PnPInk can connect to **large online icon sets (200k+ icons)** that you can **call by name**. No manual importing--just type the name.
- **Use cases:** Resource symbols, status icons, action markers, rarity pips--**instantly consistent** across the whole project.

6.10.7 Snippets & variables (write once, reuse everywhere)

- **Variables:** In text fields, you can insert **placeholders** tied to your data (names, numbers, colors, costs, etc.). When you generate, they're replaced automatically.
- **Snippets:** Think of them as **reusable text chunks** or micro-templates. Example use cases:
 - A standardized **rules block** you reuse on multiple cards.
 - A **cost line** that composes icons + numbers based on data.
 - Language variants or conditional content (e.g., add a keyword only if a value exists).
- **Benefit:** You keep your template **clean and consistent**, even for complex sets.

6.10.8 Asset sources (images, art, symbols)

PnPInk can pull artwork and symbols from multiple places:

- **Local files** you link in your data (e.g., art=images/dragon_03.png).
- **Document symbols** (shared elements duplicated across cards).
- **Sprite sheets or multi-page packs** (see "wrappers" below).
- **Online icon/name catalogs** (call by name, automatically resolved).

Tip: Always keep a simple **asset folder** structure; name files clearly to match your rows.

6.10.9 Spritesheets and virtual sources (advanced, optional)

- **Spritesheets** let you cut a bitmap into a grid and reference frames by index.
- **Virtual sources** let you reference web catalogs (Wikimedia Commons, Pixabay, Openclipart) using [wkmc://](#), [pxby://](#), or [oclp://](#), including [File:](#) and [Category:](#) modes on Wikimedia plus size targets like [large](#), [1000](#), or [1000x1000](#).
- This is ideal for **large libraries** or external packs where manual importing would be tedious.

6.10.10 Style control without limits

- Because everything happens **inside Inkscape**, you have the full power of:

- **Gradients, filters, blends, masks,**
- **Layers and symbols,**
- **Precise vector paths and text styling.**
- PnPInk simply **automates** the boring part; you keep **artistic control** at all times.

6.10.11 Professional output (CMYK & print standards)

- Export **PDFs with CMYK color, ICC profiles,** and optional **PDF/X** compliance for offset printing.
- Add **bleed, crop marks, and safe margins** automatically.
- Keep the editable **SVGs** for future updates, translations, or expansions.

6.10.12 Typical end-to-end flow (checklist)

1. Open Inkscape -> confirm **PnPInk menus** are visible.
2. Create your **template** (card/board) and assign **clear IDs**.
3. Prepare your **Google Sheet or CSV**:
4. One **row per card** (or copies).
5. Columns named after your **IDs** + any extra properties (e.g., icon, art, rarity, color, bg, copies, layout, etc.).
6. Choose a **layout** (paper size, grid, margins).
7. Use **Anchor & Fit** for placement and scaling.
8. Write **inline icons** right inside text by **name**.
9. Use **snippets/variables** for reusable blocks and smart content.
10. (Optional) Connect to **online icon libraries** and **wrappers** for multi-page asset packs.
11. **Generate** -> review pages -> export **PDF (CMYK)** for print.

7 Basic Workflow

This chapter is a practical "first successful run" guide. If you are starting from zero, complete it once end-to-end before reading full DSL reference pages.

7.1 What you need

Before running DeckMaker, confirm these prerequisites:

- Inkscape installed.
- PnPInk extension installed and visible in the Inkscape menu.

7.2 Create a Template

This step defines the visual source that all generated instances will clone.

Draw one card (or tile) as a normal SVG group.

Recommended:

- Create a rect that defines the card size (this is the **template bbox**).
- Give IDs to all elements you want to drive from data.

Inkscape ID workflow:

See [Introduction -> How IDs connect data to graphics](#).

7.3 Prepare the Dataset

This step connects table columns to SVG IDs so each row can produce one variation.

Create a CSV or Google Sheet where:

- Column A contains the template bbox id.
- Column B+ contains IDs that match your SVG objects.

If using Google Sheets, set [Google Sheet Id](#) and optional [Range](#) in DeckMaker. See [Dataset Format and Sources](#).

Example:

```
card_bbox,title,cost,art
,Fireball,3,images/fireball.png
,Shield,2,images/shield.png
```

7.4 Define Page and Layout

This step controls where generated instances are placed and how many fit per page.

Put a page/layout preset in the first cell of column A:

```
{A4 b=[-5]} L{p=3x3 g=2}
```

You can also put it in a control-only row:

```
{A4 b=[-5]} L{p=3x3 g=2},,
,Fireball,3,images/fireball.png
```

If columns B+ are empty in that row, no card is generated for that line.

This means:

- A4 page with a 5 mm inward margin.
- 3x3 grid with 2 mm gaps.

7.5 Run DeckMaker

This is the generation step: template + dataset + page/layout become output pages.

In Inkscape, run [Extensions > PnPInk DEV > DeckMaker](#) and select the dataset.

Result:

- PnPInk clones the template for each row.
- Applies Fit/Anchor and Sources.
- Creates pages and exports layouts ready for print.

7.6 Useful Inkscape Panels for PnPInk

Inkscape provides right-side dock panels that are very useful when working with PnPInk. The most relevant ones are:

- [Object > Layers and Objects...](#) (Shift+Ctrl+L): inspect object tree, layer/group hierarchy, and Z-order.
- [Object > Object Properties...](#) (Shift+Ctrl+O): edit object metadata and properties that affect matching and behavior.
- [Object > Symbols](#) (Shift+Ctrl+Y): inspect symbol definitions used by Source and icon workflows.
- [Object > Fill and Stroke](#) (Shift+Ctrl+F): review visual style values that can influence readability and print output.
- [Object > Transform...](#) (Shift+Ctrl+M): apply deterministic transforms before generation if geometry is inconsistent.
- [Object > Align and Distribute](#) (Shift+Ctrl+A): enforce consistent alignment in templates before cloning.

For ID-specific inspection/editing panels, use the single reference in [Introduction -> How IDs connect data to graphics](#).

8 Snippets

Snippets are reusable text templates expanded before normal `${var}` replacement. This feature is designed to keep datasets readable when text patterns repeat.

8.1 What snippets are

Snippets are a compact mini-language inside PnPink. They let you define short aliases that expand into longer fragments (text, SVG-rich text, paths, URL fragments, style fragments, and reusable token patterns).

Snippets are always processed first, immediately after DeckMaker reads dataset values and before normal variable substitution.

8.2 Syntax

This section defines the only supported definition format in the current engine.

Define snippets in comment lines:

```
# :Name(arg1 arg2=default) = template text
```

Rules from the current engine:

- No space is allowed between snippet name and `(`.
- Parameters are space-separated.
- Defaults use `name=value`.
- The replacement template is plain text after `=`.
- Old `->` definitions are not supported.

Call syntax:

```
:Name(value1 value2)
```

8.3 Arguments and Expansion

Argument mapping rules are what make snippets usable in real datasets.

Snippets support positional args, named args, defaults, and nesting.

```
# :Join(a b) = ${a}/${b}
# :Tf(text font size) = <tspan font-family='${font}' ${size? font-size='${size}'}>${text}</tspan>

:Join(cards heroes)          -> cards/heroes
:Tf(Title Noto 16px)         -> <tspan font-family='Noto' font-size='16px'>Title</tspan>
:Tf(Title Noto)              -> <tspan font-family='Noto'>Title</tspan>
:Tf(text=Title font=Noto size=12) -> named arguments are valid
```

Conditional blocks use `${var? ...}` and render only when `var` is non-empty.

If a snippet has exactly one parameter, that parameter captures the full inner text:

```
# :Bold(text) = <b>${text}</b>
:Bold(This is bold text) -> <b>This is bold text</b>
```

8.4 Defaults and Optional Parts

Default values are used when a parameter is omitted.

```
# :Box(text color=black) = <rect stroke='${color}'/><text>${text}</text>

:Box(Title)      -> <rect stroke='black'/><text>Title</text>
:Box(Title red)  -> <rect stroke='red'/><text>Title</text>
```

8.5 Quoting, Spaces, and Escaping

- Parameters containing spaces must use quotes.
- Use `\:` to keep a call literal.
- If a snippet does not exist, the call stays literal.
- If a call is malformed, it stays literal.

```
:Join("My Folder" file.svg) -> My Folder/file.svg
\:Join(a b)                  -> :Join(a b)
```

8.6 Nesting

Snippets can call other snippets and are expanded inside-out.

```
# :B(text) = <b>${text}</b>
# :C(text color) = <span fill='${color}'>${text}</span>

:C(:B(Name) red) -> <span fill='red'><b>Name</b></span>
```

8.7 Practical SVG Text Helpers

These helpers are optional, but they are the fastest way to build rich text consistently.

Useful helpers for `<tspan>` rich text:

```
# :Tb(text) = <tspan font-weight='bold'>${text}</tspan>
# :Ti(text) = <tspan font-style='italic'>${text}</tspan>
# :Ts(text) = <tspan text-decoration='underline'>${text}</tspan>
# :Tx(text) = <tspan text-decoration='line-through'>${text}</tspan>
# :Tc(text fill border bordercolor) = <tspan fill='${fill}'${border? stroke='${bordercolor}' stroke-width='${border}' paint-order='stroke'}>${text}</tspan>
# :Te(text) = <tspan font-family='Noto Color Emoji'>${text}</tspan>
```

Nested expansion is supported and resolved from inner to outer calls.

8.8 Processing Order

Processing order matters when snippets and `${var}` are combined in the same cell.

For each dataset cell:

1. Snippets are expanded.
2. `${var}` replacements are applied.
3. DSL placement and rendering run.

8.9 Summary Table

Concept	Syntax	Example	Result
Definition	<code># :Name(a b=def) = text</code>	<code># :Hello(name) = Hi \${name}</code>	defines snippet
Use	<code>:Hello(world)</code>		Hi World
Default	<code>param=value</code>	<code># :Box(t color=red)</code>	uses red if omitted
Conditional	<code>\${p? text}</code>	<code>\${size? font-size='\${size}'}</code>	adds only if defined
Escape	<code>\Name()</code>		prevents expansion
Nested	<code>:A(:B(x))</code>		inner first
One parameter	<code>:Bold(any text)</code>		captures all
Quoted values	<code>:Join("My Folder" file)</code>		handles spaces

8.10 Design Principles

- Consistent: follows the same spacing and escaping rules as the rest of PnPink.
- Readable: complex SVG text becomes short and maintainable.
- Composable: snippets can be nested and combined incrementally.
- Deterministic: plain text substitution rules, no runtime scripting.

9 Presets

This page is a lookup reference for standard page and shape identifiers accepted by the parser.

9.1 Page Presets (mm)

Preset	Size (mm)
A2	420 x 594
A3	297 x 420
A4	210 x 297
A5	148 x 210
A6	105 x 148
Letter	215.9 x 279.4
Legal	215.9 x 355.6
Tabloid	279.4 x 431.8

9.2 Shape Presets

Use these identifiers in `Layout{shape=...}` / `L{s=...}` and related shape-aware workflows.

Identifier	Aliases	Value (mm)
Standard	poker, magic, estandard, estandar	63 x 88
2.5x3.5inch	-	63.5 x 88.9
XL_Poker	xlpoker, xlpoker_, xlstandard, xlstandar, xl_standard, xl_standar	70.875 x 99.0
USA	bridge	56 x 87
Euro	mini	59 x 92
Asia	chimera	57.5 x 89
miniEuro	euromini	45 x 68
miniAsia	asiamini, minichimera, chimeramini	43 x 65
miniUSA	usamini	41 x 63
Tarot	-	70 x 120
FrenchTarot	-	61 x 112
Volcano	-	70 x 110
Wonder	-	65 x 100
Spanish	baraja	61 x 95
Desert	-	50 x 65
squareS	-	50 x 50
square	-	70 x 70
squareL	-	100 x 100
Dixit	-	80 x 120
CreditCard	creditcard	54 x 85.6
CR80	cr80	54 x 85.6
ID-1	id1, id-1	54 x 85.6

Also accepted (normalization rules):

- Matching is case-insensitive.
- Accents are normalized.
- Spaces, hyphens, underscores, and dots are ignored.

Examples:

- `STANDARD, standard, StAnDaRd` -> `Standard`
- `mini usa, mini-usa, mini_usa` -> `miniUSA`
- `id1, ID-1, id_1, id.1` -> `ID-1`
- `xl-poker, XL_POKER, xl poker` -> `XL_Poker`